

MODULE 14

EXPLOITATION FUNDAMENTALS

Expanded Edition - CVE research, PoC trust, controlled exploitation, Metasploit, sessions, transfer, telemetry & evidence

Authorized-lab only

Semua hands-on aktif di module ini dibatasi ke localhost / deliberately vulnerable training environment. Jangan menjalankan PoC atau exploit terhadap sistem publik, kantor, kampus, atau aset pihak lain tanpa authorization tertulis.

Omarchy / Arch Linux Edition

Project: VaxXinE/red-teaming

Version 1.0 Expanded | 05 Oktober 2026

Field	Value
Primary OS	Omarchy / Arch Linux
Prerequisite	Modules 00-13
Core tools	Metasploit Framework, Exploit-DB/SearchSploit, Python, curl, netcat, sha256sum
Hands-on model	Loopback-only simulators + non-exploit Metasploit auxiliary module
Lab count	14 guided labs + 1 capstone challenge
Next module	15 - Passwords & Credential Security

1. Learning Objectives & Safety Contract

- [] Membedakan vulnerability presence, exploitability, exploit success, dan business impact.
- [] Melakukan CVE/advisory/PoC research dengan urutan sumber yang masuk akal.
- [] Membaca source PoC sebelum execution dan mengidentifikasi dangerous behavior.
- [] Memvalidasi precondition: version, feature, auth, architecture, reachability, mitigations.
- [] Menggunakan SearchSploit sebagai offline research index - bukan trust anchor.
- [] Menggunakan Metasploit secara disiplin: search -> info -> options -> check/run -> evidence.
- [] Memahami bind/reverse/session secara konseptual tanpa membangun remote shell arbitrary.
- [] Melakukan transfer artifact dan SHA-256 verification.
- [] Melakukan failure analysis, patched retest, cleanup, dan finding documentation.

Safety contract

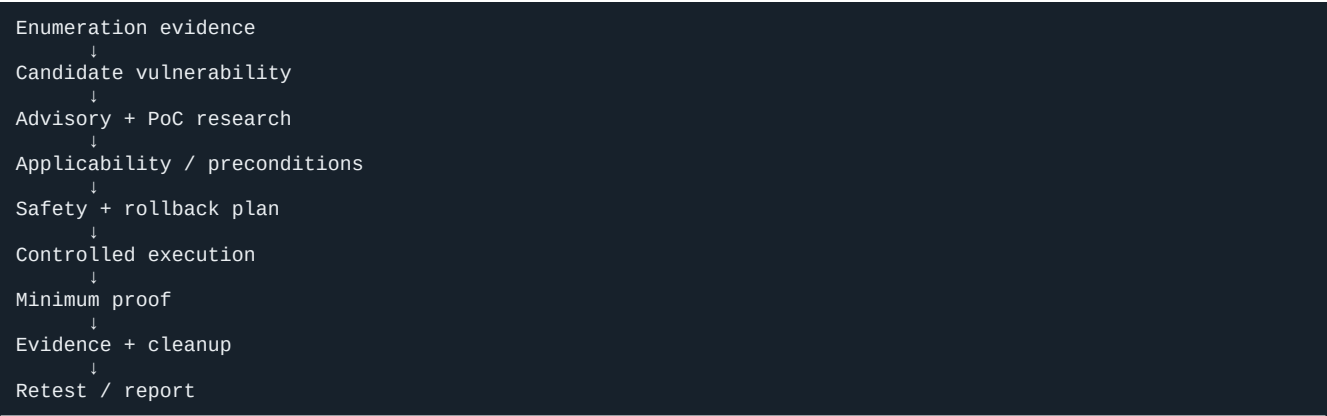
Kita memakai minimum sufficient proof. Bila proof kecil sudah membuktikan impact, jangan menaikkan blast radius hanya karena exploit mampu melakukan lebih banyak.

Prerequisite knowledge

- Module 08: service/version enumeration dan manual protocol verification.
- Module 09: CVE/CWE/CVSS, applicability, false positive, remediation.
- Module 10-13: Burp/web/API testing discipline dan evidence workflow.
- Module 05: Python/Bash fundamentals untuk memahami PoC source.

2. Exploitation Mental Model

Eksplorasi adalah eksperimen keamanan terkontrol. Targetnya bukan "mendapat shell sebanyak mungkin", tetapi membuktikan hypothesis dengan perubahan state sekecil mungkin, lalu mengembalikan environment ke kondisi yang disepakati.



Gate	Pertanyaan sebelum lanjut
Authorization	Asset, timeframe, dan technique explicitly allowed?
Applicability	Product/version/configuration benar affected?
Access	Perlu auth, user role, network position, local access?
Platform	OS/arch/runtime/library sesuai PoC?
Mitigation	Patch, hardening, WAF, sandbox, ASLR, feature flag mengubah hasil?
Blast radius	Crash, reboot, data change, account lock, outbound traffic?
Rollback	Bagaimana state dipulihkan?
Evidence	Apa proof paling kecil yang cukup?

3. CVE, Advisory, PoC, Exploit & Payload

Istilah	Arti kerja
CVE	Identifier untuk vulnerability tertentu; bukan exploit code.
CWE	Class/jenis weakness; membantu memahami root cause.
Vendor advisory	Affected version, prerequisites, fix, workaround, product context.
PoC	Code/steps untuk membuktikan vulnerability; kualitas dan safety bervariasi.
Exploit	Technique/code yang memanfaatkan weakness untuk security impact.
Payload	Aksi yang dijalankan setelah exploit path berhasil; konsepnya terpisah dari vulnerability.
Session	Channel/interaksi pasca-exploit yang dikelola tool/framework.

Jangan lompat dari banner ke exploit

Version string bisa salah, backported patch bisa ada, feature vulnerable mungkin disabled, dan product fork bisa memakai version label sama dengan behavior berbeda.

Evidence hierarchy untuk research

Prioritaskan vendor/upstream advisory dan primary technical documentation. NVD/CVE membantu indexing/enrichment. CISA KEV memberi signal observed exploitation. Exploit archive/PoC repository membantu memahami exploitability, tetapi source tetap harus direview.

4. CVE Research Worksheet

Lab 4.1 tidak membutuhkan exploit nyata. Pilih satu CVE dari produk training yang familiar, lalu isi worksheet. Tujuannya membangun kebiasaan riset sebelum eksekusi.

```
python labs/research-note.py \  
  --id CVE-2021-XXXXX \  
  --product 'Training Product' \  
  --observed-version '1.2.3' \  
  --source 'https://vendor.example/advisory' \  
  --output module14-exploitation/01-research/CVE-2021-XXXXX.md
```

- ☐ Affected product dan version range dari source primer.
- ☐ Attack vector / network position.
- ☐ Authentication/privilege prerequisite.
- ☐ Feature/configuration yang harus aktif.
- ☐ OS/architecture/runtime dependency.
- ☐ Known side effect: crash/write/restart/outbound callback.
- ☐ Patch/fixed version/workaround.
- ☐ Apakah exploit diperlukan untuk membuktikan risk, atau evidence non-destructive sudah cukup?

Decision point

Jika advisory + local configuration sudah membuktikan vulnerable state dan RoE membatasi exploitation, berhenti. Assessment yang baik tidak memaksakan exploit hanya untuk mendapatkan screenshot dramatis.

5. PoC Trust & Static Source Review

PoC adalah untrusted code. Source review minimal mencari proses execution, shell invocation, network callbacks, file writes/deletes, dynamic loading, encoded blobs, privilege changes, persistence, dan hard-coded external endpoints.

Lab 5.1 - Static helper

```
cd modules/14-exploitation-fundamentals/labs
python poc-audit.py mock-exploit.py
less mock-exploit.py
```

`poc-audit.py` tidak mengeksekusi file. Ia hanya menandai pattern yang pantas dilihat manual.

Review checklist

Cari	Contoh concern
Process execution	subprocess, os.system, shell=True, PowerShell/cmd invocation
Network	socket, requests, hard-coded callback domain/IP
Filesystem	open write/append, chmod, unlink, recursive delete
Dynamic behavior	eval/exec, importlib, ctypes, dlopen
Encoded blobs	base64/fromhex yang menyembunyikan behavior
Privilege/persistence	sudo, service/task/registry/cron modifications
Dependency install	pip/npm/curl sh style bootstrap tanpa pinning

Pattern match bukan verdict
Tidak ada hit bukan berarti PoC aman. Ada hit juga tidak otomatis berarti malicious. Static helper hanya mempercepat manual reasoning.

6. Exploit Applicability: Dari "Cocok Versi" ke Decision

Exploitability bergantung pada lebih dari version string. Module ini menyertakan helper sederhana untuk melatih reasoning berdasarkan evidence yang kita tulis sendiri.

Lab 6.1 - Target facts

```
cat > module14-exploitation/01-research/target.json <<'EOF'
{
  "product": "MockService",
  "version": "1.0",
  "os": "linux",
  "arch": "x86_64",
  "authenticated": false,
  "feature_enabled": true,
  "reachable": true,
  "scope_authorized": true
}
EOF
```

Lab 6.2 - Candidate requirements

```
cat > module14-exploitation/01-research/candidate.json <<'EOF'
{
  "product": "MockService",
  "affected_versions": ["1.0"],
  "os": ["linux"],
  "arch": ["x86_64"],
  "requires_auth": false,
  "requires_feature": true
}
EOF

python labs/applicability-check.py \
  module14-exploitation/01-research/target.json \
  module14-exploitation/01-research/candidate.json
```

Helper exact-match ini bukan CVE scanner. Tujuannya memaksa kita menyatakan assumption secara eksplisit dan membedakan PASS, FAIL, serta UNKNOWN.

7. SearchSploit / Exploit-DB Workflow

SearchSploit adalah offline search interface ke Exploit-DB. Gunakan untuk menemukan referensi, membaca file, dan membuat working copy. Jangan menjadikan keberadaan entry sebagai bukti target exploitable.

```
omarchy pkg add exploitdb

searchsploit --help
searchsploit --cve 2021-44228
searchsploit apache 2.4
searchsploit -p <EDB-ID>
searchsploit -x <path-from-results>
```

Lab 7.1 - Research without execution

- [] Cari berdasarkan product + major/minor version.
- [] Baca title, platform, type, author, date.
- [] Buka source dengan `-x` atau path hasil pencarian.
- [] Catat precondition dan side effect dari source/comments.
- [] Cari advisory primer yang mendukung applicability.
- [] Jangan execute file sampai source review selesai dan lab target tersedia.

Working copy

Bila perlu modifikasi untuk lab, copy file ke workspace dan hash original terlebih dahulu. Jangan edit satu-satunya copy sehingga provenance hilang.

```
sha256sum candidate.py > candidate.py.sha256
cp candidate.py candidate-lab.py
# edit candidate-lab.py only after review
```

8. Metasploit Architecture & Operator Discipline

Metasploit adalah framework, bukan tombol exploit. Operator tetap bertanggung jawab memahami module, options, target, payload, check behavior, side effect, session, dan cleanup.

Komponen	Fungsi
auxiliary	Scanner/helper/non-session actions.
exploit	Logic yang memanfaatkan vulnerability.
payload	Action/agent yang dijalankan setelah exploit path berhasil.
post	Action terhadap session yang sudah ada.
datastore	Option values seperti RHOSTS/RPORT/LHOST.
session	Channel hasil module/payload.
workspace/database	Mengorganisasi data bila database dikonfigurasi.

Lab 8.1 - Safe local auxiliary module

```
python -m http.server 8141 --bind 127.0.0.1
```

Terminal lain:

```
msfconsole -q
search type:auxiliary http_version
use auxiliary/scanner/http/http_version
show info
show options
set RHOSTS 127.0.0.1
set RPORT 8141
run
back
exit
```

Rule

Sebelum `run` pada module apa pun: `info`, `show options`, target verification, dan side-effect reasoning dulu. Gunakan `check` bila module menyediakan check yang sesuai dan authorized.

9. Payload, Session, Bind & Reverse - Konsep yang Harus Jelas

Konsep	Mental model	Hal yang bisa menggagalkan
Bind	Target membuka listener; tester connect masuk.	Inbound firewall, NAT, host firewall.
Reverse	Target connect keluar ke listener tester.	Egress filtering, DNS/proxy policy, wrong listener.
Staged	Komponen kecil memuat stage lanjutan.	Additional transfer/network dependency.
Stageless	Payload lengkap dikirim sekaligus.	Ukuran lebih besar / delivery constraints.
Session	Framework-managed interaction channel.	Transport drop, timeout, target process exit.

Tidak ada arbitrary shell di lab ini

Kita memodelkan arah reverse connection dengan JSON facts-only callback. Listener tidak menerima command dan agent tidak mengeksekusi instruction dari network.

Blue-team perspective

- Bind: new listening socket/process association dapat terlihat.
- Reverse: outbound connection dari process yang tidak biasa dapat terlihat.
- Session activity: process tree, network telemetry, file creation, service changes, dan auth logs dapat menjadi detection surface.

10. Lab Environment & Baseline

Lab 10.1 - Workspace

```
cd modules/14-exploitation-fundamentals/labs
./workspace-init.sh
```

Workspace:

```
module14-exploitation/
├── 00-scope/
├── 01-research/
├── 02-poc-review/
├── 03-evidence/
├── 04-transfer/
├── 05-cleanup/
└── 06-report/
```

Lab 10.2 - Start mock target

```
python mock-vuln-service.py --mode vulnerable
```

Service hanya bind ke `127.0.0.1:8140`. Terminal lain ambil baseline:

```
printf 'STATUS\n' | nc 127.0.0.1 8140
ss -ltnp | grep 8140
```

Baseline first

Catat service identity dan expected normal behavior sebelum exploit test. Tanpa baseline, kita sulit menjelaskan apa yang benar-benar berubah.

11. Controlled Exploitation of the Simulator

Lab 11.1 - Audit PoC

```
python poc-audit.py mock-exploit.py
sha256sum mock-exploit.py
less mock-exploit.py
```

Lab 11.2 - Execute localhost-only PoC

```
python mock-exploit.py \
  --host 127.0.0.1 \
  --port 8140 \
  --evidence-dir module14-exploitation/03-evidence
```

Expected proof berupa `SIMULATED-IMPACT`, correlation ID, dan training UID. Tidak ada OS command execution atau shell.

Lab 11.3 - Guardrail test

```
python mock-exploit.py --host 192.168.1.10 --port 8140
```

Expected: helper menolak target non-loopback sebelum koneksi dibuat.

Minimum proof

Correlation ID + request metadata + response sudah cukup untuk simulator. Menambah capability tidak menambah nilai finding.

12. Failure Analysis & Patched Retest

Eksplorasi sering gagal. Failure bukan sinyal untuk langsung mencoba exploit lain; failure harus menghasilkan hipotesis baru.

Symptom	Kemungkinan penyebab	Next safe check
Connection refused	Service down/wrong port/listen address	`ss`, baseline request, process state.
Timeout	Filtering, wrong path, service hang	Reachability + protocol-level baseline.
Exploit returns error	Version/config/precondition mismatch	Re-read advisory + module info/source.
No session/callback	Wrong listener/egress/process exited	Listener state + outbound policy + logs.
Works in lab, not target	Backport/hardening/architecture differences	Compare exact environment facts.
Crash/restart	PoC side effect / unstable target	Stop; follow RoE escalation and cleanup.

Lab 12.1 - Patched mode

Stop vulnerable simulator lalu jalankan:

```
python mock-vuln-service.py --mode patched
```

Ulangi PoC yang sama:

```
python mock-exploit.py --host 127.0.0.1 --port 8140
```

Expected: vulnerable proof tidak muncul. Catat sebelum/ sesudah sebagai miniature remediation validation.

Retest discipline

Gunakan input yang sama ketika membandingkan vulnerable vs patched state. Ubah satu variable: mitigation. Ini membuat evidence lebih kuat.

13. Reverse-Connection Model - Safe Training

Lab 13.1 - Listener

```
python callback-listener.py --port 8143
```

Lab 13.2 - Training agent

```
python training-agent.py --host 127.0.0.1 --port 8143
```

Listener menerima JSON facts seperti timestamp, platform, Python version, PID, dan mode training. Listener tidak mengirim command balik.

```
training-agent
  | outbound TCP
  v
127.0.0.1:8143 listener
  |
  └─ fixed JSON facts only
```

Apa yang harus lo jawab

- [] Mana yang menjadi listener?
- [] Arah koneksi siapa ke siapa?
- [] Jika listener salah port, failure terlihat seperti apa?
- [] Jika outbound policy memblok koneksi, apa telemetry yang mungkin ada?
- [] Apa bedanya konsep ini dengan bind model?

14. File Transfer, Artifact Provenance & Integrity

File transfer sering dianggap langkah kecil, padahal salah artifact atau binary berubah di tengah workflow dapat membuat hasil tidak reproducible.

Lab 14.1 - Local HTTP transfer

```
mkdir -p /tmp/module14-transfer
printf "training artifact\n" > /tmp/module14-transfer/artifact.txt
sha256sum /tmp/module14-transfer/artifact.txt

python -m http.server 8142 \
  --bind 127.0.0.1 \
  --directory /tmp/module14-transfer
```

Terminal lain:

```
curl --fail --silent --show-error \
  http://127.0.0.1:8142/artifact.txt \
  -o module14-exploitation/04-transfer/artifact.txt

sha256sum module14-exploitation/04-transfer/artifact.txt
```

Lab 14.2 - Verify automatically

```
python transfer-verify.py \
  /tmp/module14-transfer/artifact.txt \
  module14-exploitation/04-transfer/artifact.txt
```

Production habit

Catat source, destination, checksum, timestamp, purpose, dan cleanup path. Untuk real engagement, approved transfer channel dan data classification juga penting.

15. Exploit Modification Basics - Safe Scope

Modifikasi exploit tidak berarti menambah offensive capability. Untuk pemula, fokus pada memahami interface code: target address, port, timeout, protocol framing, expected response, dan logging.

Lab 15.1 - Working copy

```
cp mock-exploit.py \
  module14-exploitation/02-poc-review/mock-exploit-lab.py

sha256sum mock-exploit.py \
  module14-exploitation/02-poc-review/mock-exploit-lab.py
```

Latihan perubahan yang aman

- [] Ubah default timeout dari 2.0 ke 3.0 detik.
- [] Tambahkan label operator ke evidence JSON.
- [] Tambahkan explicit validation untuk rentang port 1024-65535.
- [] Jangan menghapus loopback guard.
- [] Jalankan unit-like check terhadap vulnerable dan patched simulator.
- [] Bandingkan diff dengan original sebelum menyimpan hasil.

```
diff -u mock-exploit.py \
  module14-exploitation/02-poc-review/mock-exploit-lab.py
```

Trust boundary

Kalau modifikasi membutuhkan disable safety guard atau memperluas target ke Internet, itu bukan lagi exercise module ini.

16. Evidence, Activity Log & Cleanup

Lab 16.1 - Activity log

```
python activity-log.py \
  module14-exploitation/03-evidence/activity.jsonl \
  --action "mock exploit test" \
  --target "127.0.0.1:8140" \
  --result "simulated impact observed"
```

Lab 16.2 - Evidence manifest

```
./evidence-manifest.sh module14-exploitation/03-evidence

cd module14-exploitation/03-evidence
sha256sum -c evidence-manifest.sha256
```

Lab 16.3 - Cleanup verification

Stop manual servers/listeners, hapus temp artifact, lalu:

```
python cleanup-check.py --ports 8140 8141 8142 8143
rm -rf /tmp/module14-transfer
```

Cleanup helper hanya mengecek apakah training ports masih menerima koneksi di loopback.

Log item	Contoh
What	PoC simulator execution
When	UTC timestamp
Where	127.0.0.1:8140
Result	SIMULATED-IMPACT correlation=...
Artifacts	JSON evidence + SHA-256
Cleanup	Server/listener stopped; temp files removed
Retest	Patched mode no longer returns impact

17. Detection-Aware Exploitation

Red Teamer yang matang memahami telemetry yang ditimbulkan tindakannya. Tujuannya bukan bypass dulu; tujuannya mengetahui apa yang defender dapat lihat.

Action	Possible telemetry
Service probing	Connection logs, firewall events, application access logs.
PoC request	Unusual request length/pattern, protocol errors, crash logs.
Process launch	Process creation, parent/child chain, command-line telemetry.
Reverse callback	Outbound connection, DNS, proxy/firewall log, process-network correlation.
File transfer	HTTP access log, file creation, hash/EDR telemetry.
Privilege change	Auth logs, token/group changes, service/task events.
Cleanup	File deletion/process exit - juga bisa meninggalkan telemetry.

Red vs Blue

Jangan belajar exploitation terpisah dari observability. Di module lanjutan kita akan kembali ke Sysmon/Wazuh/EDR telemetry; sekarang cukup biasakan menanyakan "aksi gue terlihat di mana?"

Stop conditions

- Unexpected production-like data muncul.
- Service crash/restart yang tidak direncanakan.
- Target ternyata out-of-scope atau ownership meragukan.
- PoC melakukan outbound connection di luar lab.
- PoC membutuhkan destructive/persistent step yang tidak ada di RoE.

18. Capstone - Unknown Training Target

Jangan buka source `mock-vuln-service.py` sampai challenge selesai. Gunakan hanya loopback training service.

- [] Buat workspace dan scope note.
- [] Fingerprint service dengan baseline request.
- [] Tulis hypothesis vulnerability berdasarkan evidence.
- [] Isi research worksheet untuk candidate fictional vulnerability.
- [] Jalankan applicability-check dan jelaskan PASS/FAIL/UNKNOWN.
- [] Audit PoC source; catat minimal lima observasi.
- [] Hash original PoC sebelum working-copy modification.
- [] Execute PoC pada vulnerable mode dan simpan minimum proof.
- [] Ulangi test pada patched mode dengan input yang sama.
- [] Lakukan reverse-connection model dan jelaskan arah koneksi.
- [] Transfer satu harmless artifact dan verifikasi SHA-256.
- [] Generate evidence manifest + activity log.
- [] Cleanup semua training listeners dan temporary files.
- [] Tulis finding: precondition -> proof -> impact -> remediation -> retest.

Scoring rubric

Area	Bobot
Safety/scope discipline	20%
Research & applicability	20%
PoC source reasoning	20%
Controlled execution + retest	20%
Evidence, cleanup, reporting	20%

19. Knowledge Check & Common Failure Modes

1. Apa beda vulnerability presence dan exploitability?
2. Apa beda CVE, advisory, PoC, exploit, payload, dan session?
3. Mengapa version string tidak cukup sebagai exploit decision?
4. Sebutkan minimal enam hal yang harus diperiksa pada source PoC.
5. Mengapa PoC yang bekerja di lab bisa gagal di target dengan version sama?
6. Apa kegunaan SearchSploit dan apa yang tidak dibuktikannya?
7. Apa urutan aman sebelum `run` di Metasploit?
8. Apa beda bind dan reverse connection?
9. Apa beda staged dan stageless secara konseptual?
10. Mengapa minimum sufficient proof lebih baik daripada proof agresif?
11. Mengapa checksum penting pada file transfer?
12. Apa yang harus ada di cleanup log?
13. Mengapa patched retest menggunakan input yang sama?
14. Apa contoh telemetry dari exploit attempt?

Anti-pattern cepat

Anti-pattern	Perbaiki
Version match -> auto exploit	Validate advisory + configuration + precondition.
Run PoC sebagai root	Least privilege + isolated lab.
Trust repository popularity	Source review + provenance + hash.
Disable safety guard agar "berhasil"	Perbaiki lab atau assumption; jangan perluas scope.
No raw evidence	Simpan request/output/timestamp/hash.
No cleanup plan	Rollback ditentukan sebelum execution.
Exploit gagal -> random exploit lain	Failure analysis berbasis hypothesis.

20. Cheat Sheet, Exit Criteria & References

Cheat sheet

```
# Packages
omarchy pkg add metasploit exploitdb

# SearchSploit - research only
searchsploit --cve <CVE-ID>
searchsploit <product> <version>
searchsploit -x <path-from-results>

# Metasploit safe workflow
msfconsole -q
search type:auxiliary http_version
use auxiliary/scanner/http/http_version
show info
show options
set RHOSTS 127.0.0.1
set RPORT 8141
run
back

# Training PoC workflow
python labs/poc-audit.py labs/mock-exploit.py
python labs/mock-vuln-service.py --mode vulnerable
python labs/mock-exploit.py --host 127.0.0.1 --port 8140

# Integrity + cleanup
sha256sum <artifact>
./labs/evidence-manifest.sh module14-exploitation/03-evidence
python labs/cleanup-check.py --ports 8140 8141 8142 8143
```

Exit criteria

- [] Bisa menjelaskan exploit decision tanpa bergantung pada version string saja.
- [] Bisa audit source PoC dan menyebut dangerous operations yang perlu diperiksa.
- [] Bisa menggunakan SearchSploit dan Metasploit dengan pre-run discipline.
- [] Bisa menjelaskan payload/session/bind/reverse/staged/stageless secara konseptual.
- [] Bisa membedakan exploit failure karena environment vs karena vulnerability tidak applicable.
- [] Bisa melakukan minimum proof, patched retest, evidence hashing, dan cleanup verification.

Primary references

Metasploit Documentation: <https://docs.metasploit.com/>

Arch Linux - metasploit: https://archlinux.org/packages/extra/x86_64/metasploit/

Arch Linux - exploitdb: <https://archlinux.org/packages/extra/any/exploitdb/>

Exploit Database / SearchSploit Manual: <https://www.exploit-db.com/searchsploit>

NIST National Vulnerability Database: <https://nvd.nist.gov/>

CISA Known Exploited Vulnerabilities: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

CVE Program: <https://www.cve.org/>

OffSec PEN-200: <https://www.offsec.com/courses/pen-200/>

Next: Module 15 - Passwords & Credential Security

Berikutnya kita masuk hashing, salting, offline cracking, wordlists, password policy, credential reuse, spraying concepts, MFA, dan secure credential handling di authorized lab.